MODULE 09 · SECTION 1 - DESIGNING DATA PROCESSING SYSTEMS

# Designing for Portability and Multi-cloud

Open file and table formats, Lakehouse, BigQuery Omni, portable code, and residency across clouds.

SECTION 1 of 5 EXAM WEIGHT 22% MODULE 9 of 45

admin@sthithapragna.com

Portability is not refusing to commit. It is **keeping the exit door unlocked**.

# Four layers to keep open

| LAYER | LOCKS YOU IN | KEEPS IT OPEN |
|---|---|---|
| File format | A store's own internal storage | Parquet, Avro, ORC |
| Table format | Metadata only one engine reads | Apache Iceberg |
| Catalog | A catalog only one engine queries | An Iceberg REST catalog |
| Code | APIs tied to one service | Apache Beam or Apache Spark |

THE TRAP

Open files are not enough. Without an open table format and catalog, each engine sees only loose files.

# Rows or columns?

| FORMAT | LAYOUT | BEST FOR |
|---|---|---|
| Avro | Row-based, schema inside the file | Fast appends, streaming archives, schema change |
| Parquet | Columnar, compressed | Analytics that read a few columns of many rows |
| ORC | Columnar, compressed | Hive estates, like Larkspur's history |
| CSV and JSON | Text, no enforced types | Interchange with systems that need text |

THE TIEBREAKER

Row formats write fast. Column formats read fast. Pick by what happens to the data most often.

# Apache Iceberg: the inventory list

- Files are the boxes. **Iceberg is the inventory list** that says which boxes make up the table.
- **Atomic commits**: a writer swaps in a new list, so readers never see half a change.
- **Snapshots** give time travel. **Schema evolution** adds or renames columns safely.
- **Many engines, one table**: Spark, Flink, Trino and BigQuery read the same list.

> ON THE EXAM
>
> 'Several engines must read and write the same tables, with transactions' points to Iceberg.

# BigLake is now Lakehouse

- **22 April 2026**: BigLake became **Google Cloud Lakehouse**.
- The BigLake metastore became the **Lakehouse runtime catalog**.
- **API and IAM names did not change**, and many docs pages still say BigLake.
- **BigLake tables for Apache Iceberg in BigQuery** are now **Apache Iceberg managed tables**.

> REMEMBER
>
> The exam guide still says BigLake. Answer with whichever name the question uses.

# Lakehouse tables: governed tables over open files

- **Access delegation**: a connection reads the bucket, so users need table access only.
- **Row and column security** and masking, even for Spark through the Storage API.
- Parquet, ORC, Avro, CSV, JSON, **Iceberg** and **Delta Lake**, on Cloud Storage, S3 or Azure.
- **Metadata caching** speeds up tables with many files, like Larkspur's history.

> THE SIGNAL
>
> 'Analysts must not read the bucket directly' points to a Lakehouse table, not a plain external table.

# Two kinds of Iceberg table: who writes?

ICEBERG MANAGED TABLES

## BigQuery writes

- Data in your bucket, as Parquet
- BigQuery DML, streaming, compaction
- Others read exported metadata
- Never edit the files directly

RUNTIME CATALOG TABLES

## Open engines write

- Spark, Flink or Trino write Iceberg
- Iceberg REST catalog endpoint
- BigQuery queries the same tables
- BigQuery writes are Preview

THE TIEBREAKER

Ask who writes the table. BigQuery: managed. Spark or another engine: the runtime catalog.

# An Iceberg managed table for Larkspur

```
1  CREATE TABLE curated.readings_iceberg (
2    meter_id STRING, reading_ts TIMESTAMP, kwh NUMERIC)
3  CLUSTER BY meter_id
4  WITH CONNECTION `lk-warehouse-prod.europe-west2.lake`
5  OPTIONS (
6    file_format = 'PARQUET',
7    table_format = 'ICEBERG',
8    storage_uri = 'gs://lk-lake-prod/readings');
9
10 EXPORT TABLE METADATA FROM curated.readings_iceberg;
```

4 The connection reads and writes the bucket, not the users.

6-7 Open files, open table format, in Larkspur's own bucket.

10 Publishes Iceberg metadata for Spark. Run it on a schedule.

HOW IT WORKS

BigQuery owns the writes. The files stay open, and stay in Larkspur's bucket.

# BigQuery Omni: take the engine to the data

- The BigQuery engine **runs inside AWS or Azure**, next to the data.
- AWS: N. Virginia, Oregon, Seoul, Sydney, Ireland, **Frankfurt**. Azure: East US 2.
- Queries **Lakehouse tables over S3 or Blob**. No managed tables, no DML, no BigQuery ML.
- Billed on demand or with **Enterprise edition** reservations.

> THE POINT
>
> Omni moves the compute, not the data. The question to ask is where the results go.

# Where do Omni results go?

| STATEMENT | WHAT CROSSES CLOUDS | LIMIT |
|---|---|---|
| `EXPORT DATA` | Nothing. Results stay in S3 or Blob | None crossing clouds |
| `Cross-cloud join` | The Omni side's result | 60 GB per transfer |
| `CREATE TABLE AS SELECT` | A filtered result, into BigQuery | 60 GB per transfer |
| `LOAD DATA` | Raw files, unfiltered | No limit |
| `Cross-cloud materialized view` | A continuous, incremental replica | Not for Azure Blob |

THE SIGNAL

'Results must stay in AWS' is EXPORT DATA. 'Keep a fresh copy in BigQuery' is a cross-cloud materialized view.

# The cross-cloud Lakehouse

- **Preview**: query Iceberg tables through remote catalogs. AWS Glue, Databricks Unity, Snowflake.
- Google calls it **a modern alternative to BigQuery Omni**.
- It **caches data segments inside Google Cloud** to cut egress. A cache is a copy.
- Runs over the internet, or privately over **Cross-Cloud Interconnect**.

> HONEST LIMIT
>
> Too new for most exam questions. Know it exists, and know that caching moves data.

# Residency across clouds

- **Where does processing run?** Omni runs in the source cloud's region.
- **Where do results land?** Joins and CTAS copy results into a BigQuery region.
- **Are there caches?** Caches and replicas are copies, and rules usually cover copies.
- **Who pays egress?** The source cloud bills data leaving it.

> THE TAKEAWAY
>
> Trace every copy, not just the original. That is how residency questions are won.

# Apache Beam or Apache Spark?

APACHE BEAM

## One pipeline, many runners

- Dataflow, Flink, Spark and more
- Batch and streaming in one model
- Best for new pipelines
- Dataflow adds autoscaling

APACHE SPARK

## Runs wherever Spark runs

- Managed Service for Apache Spark
- Serverless batches, or clusters
- Best for existing Spark code
- Lift and shift Hadoop jobs

THE TIEBREAKER

Existing Spark code: keep it Spark. A new pipeline that must stay portable: Beam.

# Phrases that pick the answer

| THE QUESTION SAYS | REACH FOR |
|---|---|
| **Query S3 in place**, results stay in AWS | BigQuery Omni with EXPORT DATA |
| **Several engines** share tables, with transactions | Apache Iceberg |
| Spark writes, **BigQuery reads** | Iceberg in the Lakehouse runtime catalog |
| **No bucket access** for analysts | Lakehouse table with a connection |
| **Schema changes often**, fast appends | Avro |
| **Existing Spark jobs**, minimal change | Managed Service for Apache Spark |

THE PATTERN

Open formats, one copy, and engines that come to the data.

Larkspur Energie inherited a 40 TB archive of grid sensor data, stored as Parquet in Amazon S3 in Frankfurt. Its agreement says the files must not be copied out of AWS.

Analysts in Hamburg want to query the archive with BigQuery SQL, and keep their results in the same AWS account.

## What should the team set up?

A Storage Transfer Service, copying the archive into a europe-west3 bucket

B BigQuery Omni in AWS Frankfurt, with results written out by EXPORT DATA

C The cross-cloud Lakehouse, with caching turned on to cut egress charges

D A Dataflow pipeline that reads the S3 files and streams rows to BigQuery

◆ PAUSE AND DECIDE

## What should the team set up?

A Copy with Storage Transfer

*It copies the files out of AWS, which the agreement forbids.*

B Omni, then EXPORT DATA

The query runs in AWS Frankfurt, and the results stay in S3.

C Cross-cloud Lakehouse

*Its cache stores data segments in Google Cloud, and it is Preview.*

D A Dataflow pipeline

*It streams every row out of AWS, and adds a pipeline to run.*

THE SIGNAL

'Must not be copied out' plus 'results stay in AWS' means the engine goes to the data.

> Larkspur archives every raw meter message to Cloud Storage as it arrives, so it can replay history into BigQuery after a bug.
>
> New firmware adds fields every few months, and old files must stay readable without a separate schema file.

## Which file format should the archive use?

A CSV, compressed with gzip, with a header row at the top of each file

B Parquet, written in small files as the messages arrive in the archive

C Avro, with the writer's schema stored inside every file it writes

D JSON lines, one message per line, with no schema stored anywhere

◆ PAUSE AND DECIDE

# Which file format should the archive use?

A **Compressed CSV**

*No types and no schema, and new fields shift the columns.*

B **Parquet, small files**

*Built for big analytic reads. Record-by-record writes make tiny files.*

C **Avro, schema inside**

Row-based for fast appends. The schema travels and evolves with the data.

D **JSON lines**

*Readable, but nothing enforces types, and every row repeats field names.*

THE SIGNAL

'Appends as it arrives' plus 'schema changes, no separate schema file' is Avro.

> Larkspur's settlement tables are written daily by Spark jobs on Managed Service for Apache Spark. Analysts query the same tables in BigQuery.
>
> The board wants one copy of the data in an open format, which a different engine such as Trino could use later.

## How should the tables be stored?

A Native BigQuery tables, with Spark writing through the BigQuery connector

B Iceberg managed tables in BigQuery, with Spark writing files to the bucket

C Parquet files as Hive external tables, with a nightly copy into BigQuery

D Iceberg tables in the Lakehouse runtime catalog, via its REST endpoint

◆ PAUSE AND DECIDE

## How should the tables be stored?

A **Native BigQuery tables**

*One copy, but in BigQuery's own storage, so not an open format.*

B **Managed Iceberg, Spark writes**

*Editing files outside BigQuery can lose data. BigQuery must write.*

C **Hive tables, nightly copy**

*Two copies to keep in step, and plain Parquet has no transactions.*

D **Runtime catalog Iceberg**

Spark commits Iceberg, BigQuery reads it, and Trino can join later.

THE SIGNAL

Spark writes and BigQuery reads: Iceberg through the runtime catalog. One copy, open, shared.

> Larkspur runs about 300 Spark settlement jobs on the Bristol Hadoop cluster. They must move before the lease ends on 30 June 2027.
>
> The team wants minimal code changes, as few clusters to patch as possible, and the freedom to run the jobs elsewhere later.

## What is the best plan?

A Keep the jobs as Spark, on Managed Service for Apache Spark

B Rewrite every job in Apache Beam, and run them all on Dataflow

C Rewrite every job as BigQuery SQL, scheduled through Dataform

D Install Spark on self-managed Compute Engine VMs, as it is today

◆ PAUSE AND DECIDE

# What is the best plan?

**A** Keep Spark, managed

Few code changes, serverless batches where possible, and Spark runs anywhere.

**B** Rewrite in Beam

*Portable, but three hundred rewrites against a fixed deadline.*

**C** Rewrite as BigQuery SQL

*Three hundred rewrites, and the logic is tied to one warehouse.*

**D** Self-managed Spark on VMs

*Few code changes, but every cluster is yours to patch.*

THE SIGNAL

Existing Spark code plus a deadline means keep it Spark. Spark is already portable.

# Module 09 in five lines

- Keep four layers open: **files, table format, catalog, code**.
- **Avro** for appends and changing schemas. **Parquet** or **ORC** for analytics.
- **Iceberg** shares tables. BigQuery writes: **managed**. Spark writes: **runtime catalog**.
- **Omni** moves the engine, not the data. **EXPORT DATA** keeps results in that cloud.
- **Trace every copy**: results, replicas and caches all count for residency.

COMING UP

Next: module 10, the start of section 2, planning pipelines.